

Projet AWS : orchestration ECS et Kubernetes

LOUHOU Godwill - LAKRIB Ikram - BENOIT Enzo

2026-08-28

1. Répartition du travail

- Godwill prend en charge le cadrage, le schéma d'architecture, l'application Docker et le module Terraform consacré à ECS Fargate.
- Ikram prend en charge le module Terraform Kubernetes, le HPA, les règles de sécurité et les tests de charge et d'autoréparation sur Minikube.
- Enzo prend en charge le pipeline Jenkins, l'intégration des deux modules, les tests d'idempotence et la documentation d'exécution du projet.

2. Rappel du sujet

2.1. Contexte

Une entreprise veut exécuter la même application conteneurisée sur Amazon ECS et sur Kubernetes. ECS correspond à la cible AWS. Le format Kubernetes garde le déploiement utilisable hors d'AWS. Le travail demandé consiste à automatiser les deux déploiements avec Terraform et Jenkins.

2.2. Formalisation du besoin

Le projet déploie une seule application sur deux cibles. La première est Amazon ECS Fargate dans AWS. La seconde est un cluster Kubernetes local fourni par Minikube. Terraform décrit les ressources et Jenkins exécute les opérations de validation et de déploiement.

2.2.1. Charge et disponibilité

La démonstration génère peu de trafic en temps normal. Elle doit toutefois montrer le comportement des deux plateformes lorsque la charge CPU augmente.

Chaque déploiement démarre avec deux instances, deux tâches sur ECS et deux pods sur Kubernetes. L'orchestrateur remplace une instance si elle s'arrête. L'autoscaling en ajoute lorsque la charge CPU dépasse le seuil configuré.

2.2.2. Sécurité

Les identifiants AWS ne sont pas stockés dans Git. Jenkins les conserve dans son gestionnaire de credentials et masque leurs valeurs dans les logs.

Un Security Group limite l'accès au service ECS à une seule adresse IPv4. Sur Kubernetes, le conteneur s'exécute sans privilèges. Une politique d'admission refuse aussi les images qui utilisent le tag latest.

2.2.3. Coût

Le compte AWS Academy impose de limiter les ressources. Les tâches Fargate utilisent donc la plus petite configuration retenue pour ce projet. Kubernetes fonctionne en local avec Minikube.

Le service ECS n'utilise pas de Load Balancer. Chaque tâche reçoit une adresse IP publique, ce qui suffit pour la démonstration et évite une ressource AWS supplémentaire.

2.2.4. Contraintes

Toutes les ressources AWS sont créées dans us-east-1. ECS réutilise le rôle LabRole fourni par AWS Academy, car le compte du lab interdit la création de rôles IAM.

Le déploiement Kubernetes utilise Minikube. Les permissions du lab ne permettent pas de créer un cluster EKS.

2.3. Architecture cible

Le dépôt Git contient l'application, le Dockerfile, les modules Terraform et le Jenkinsfile. Jenkins récupère le commit demandé, valide la configuration et produit un plan Terraform. Le pipeline attend ensuite une confirmation manuelle avant l'application du plan.

Terraform gère les ressources ECS et Kubernetes. Les deux cibles exécutent la même application. Le hash court du commit Git devient le tag de l'image et relie chaque déploiement à son commit.

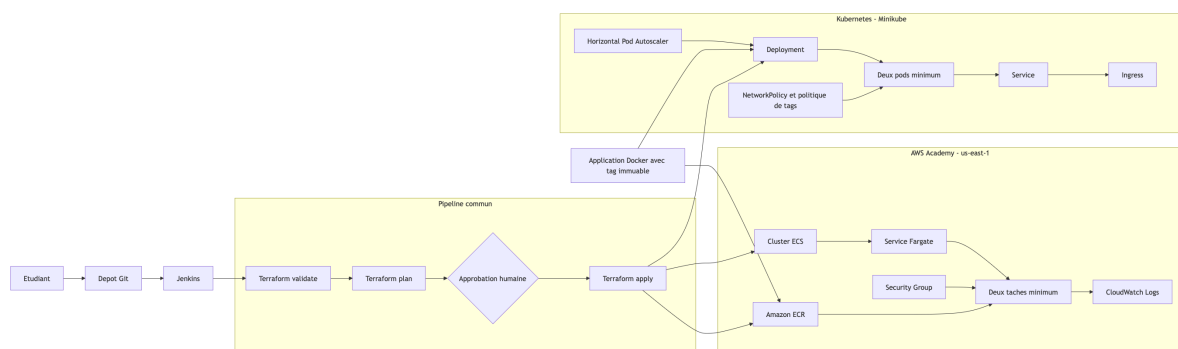


Figure 1: Schéma général : Git, Jenkins, Terraform, ECS et Kubernetes

3. Déploiement sur ECS via Terraform

3.1. Application utilisée

Nous avons écrit un petit serveur HTTP en Node.js pour tester les deux orchestrateurs. Il utilise uniquement les modules fournis avec Node.js. L'image ne contient donc aucune dépendance applicative à installer.

L'application écoute sur le port 8080 et propose trois routes :

- / renvoie le nom de l'application, sa version, la plateforme et le nom du conteneur ;
- /health répond lorsque le serveur fonctionne ;
- /cpu occupe le processeur pendant un court moment pour tester l'autoscaling.

Les variables d'environnement APP_VERSION et PLATFORM indiquent la version exécutée et la cible du déploiement. ECS et Kubernetes interrogent /health pour détecter un conteneur qui ne répond plus.

3.2. Conteneurisation de l'application

L'image Docker repose sur node:20.18.3-alpine3.21. Le tag fixe la version de Node.js et évite qu'une nouvelle image de base modifie le résultat d'un build sans changement dans le dépôt.

Le fichier Dockerfile réalise les opérations suivantes :

- création du dossier de travail /app ;
- copie du fichier server.js dans l'image ;
- ouverture du port 8080 ;
- exécution de l'application avec l'utilisateur non privilégié node ;

- lancement du serveur avec la commande `node server.js`.

Le HEALTHCHECK appelle régulièrement `/health`. Docker signale le conteneur comme défaillant si la route ne répond plus.

Nous avons d'abord construit l'image sous le nom `orchestration-demo:v1`. Le test local sur le port 8081 a permis de vérifier les trois routes avant tout déploiement.

```
→ PROJET git:(main) docker build -t orchestration-demo:v1 ./app
[+] Building 3.2s (8/8) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 389B                    0.0s
=> [internal] load metadata for docker.io/library/node:20.18.3-alpine3 0.4s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 82B                          0.0s
=> [1/3] FROM docker.io/library/node:20.18.3-alpine3.21@sha256:053c1d9 2.6s
=> => resolve docker.io/library/node:20.18.3-alpine3.21@sha256:053c1d9 0.0s
=> => sha256:6e771e15690e2fabf2332d3a3b744495411d6e0b0 3.99MB / 3.99MB 0.3s
=> => sha256:4638b6ca67cd84552a43f82ccce36e24a132cf6 42.32MB / 42.32MB 1.4s
=> => sha256:bd6deac41d88202673083931856f24d0d6a2c38c6 1.26MB / 1.26MB 0.7s
=> => sha256:053c1d99e608fe9fa0db6821edd84276277c0a663 7.67kB / 7.67kB 0.0s
=> => sha256:135ecf7e1ceb77edd269d142c8f5124787c17e70e 1.72kB / 1.72kB 0.0s
=> => sha256:2ddb39840cee49e9b863f0a82e57b1039450db06c 6.20kB / 6.20kB 0.0s
=> => extracting sha256:6e771e15690e2fabf2332d3a3b744495411d6e0b00b2ae 0.1s
=> => sha256:0dbd61e575ae4825eba984b7b77e1caab031c8e61539b 444B / 444B 0.5s
=> => extracting sha256:4638b6ca67cd84552a43f82ccce36e24a132cf6f53b407 1.0s
=> => extracting sha256:bd6deac41d88202673083931856f24d0d6a2c38c6e17ed 0.0s
=> => extracting sha256:0dbd61e575ae4825eba984b7b77e1caab031c8e61539be 0.0s
=> [internal] load build context                        0.0s
=> => transferring context: 1.18kB                      0.0s
=> [2/3] WORKDIR /app                                  0.1s
=> [3/3] COPY --chown=node:node server.js server.js    0.0s
=> exporting to image                                  0.0s
=> => exporting layers                                    0.0s
=> => writing image sha256:71c97c49ce3264f57b3f99b364516498fc17d1051bb 0.0s
=> => naming to docker.io/library/orchestration-demo:v1 0.0s
```

Figure 2: Création de l'image Docker

```
→ PROJET git:(main) ✕
docker run --rm -d \
  --name orchestration-demo-test \
  -p 8080:8080 \
  -e APP_VERSION=v1 \
  -e PLATFORM=Docker \
  orchestration-demo:v1
78ee47e6174a1e4c7027bdc4b2798b95a2f8401b4c415bd430d743808e1767af
```

Figure 3: Lancement du conteneur Docker sur le port local 8081

```

→ PROJÉT git:(main) ✗ curl http://127.0.0.1:8081/
{"application":"orchestration-demo","version":"v1","platform":"Docker","instance":"d799f26e544e"}%
→ PROJÉT git:(main) ✗ curl http://127.0.0.1:8081/health
{"status":"ok"}%
→ PROJÉT git:(main) ✗ curl http://127.0.0.1:8081/cpu
{"message":"CPU load generated"}%

```

Figure 4: Vérification des routes /, /health et /cpu

3.3. Initialisation Terraform pour AWS

Les fichiers d'infrastructure se trouvent dans le dossier terraform, séparé du code Node.js. Le provider AWS cible us-east-1. Les contraintes de version de Terraform et des providers évitent qu'une autre machine sélectionne des versions incompatibles.

Le nom du projet, le tag de l'image et l'adresse IP autorisée sont des variables. Le fichier local terraform.tfvars leur donne une valeur sur mon poste, mais Git l'ignore. Le dépôt contient seulement terraform.tfvars.example, sans donnée personnelle.

terraform init installe les providers et les modules. Nous lançons ensuite terraform validate avant chaque plan pour repérer une référence manquante ou une valeur invalide sans créer de ressource.

Création des fichiers Terraform :

versions.tf

```

terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

```

providers.tf

```

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project = var.project_name
      ManagedBy = "Terraform"
    }
  }
}

```

variables.tf

```

variable "aws_region" {
  description = "Region AWS utilisée pour le projet"
  type        = string
  default     = "us-east-1"

  validation {
    condition     = var.aws_region == "us-east-1"
    error_message = "Le projet doit être déployé dans us-east-1."
  }
}

```

```

    }
}

variable "project_name" {
  description = "Nom utilise pour identifier les ressources"
  type        = string
  default     = "ipssi-orchestration"
}

variable "image_tag" {
  description = "Tag de l'image Docker"
  type        = string
  default     = "v1"

  validation {
    condition     = var.image_tag != "latest"
    error_message = "Le tag latest est interdit."
  }
}

variable "allowed_cidr" {
  description = "Adresse IP autorisee a contacter le service ECS"
  type        = string

  validation {
    condition     = can(cidrhost(var.allowed_cidr, 0)) && endswith(var.allowed_cidr,
"/32")
    error_message = "L'adresse doit etre fournie au format x.x.x.x/32."
  }
}

terraform.tfvars
image_tag      = "v1"
allowed_cidr   = "104.28.42.20/32"

```

```
→ terraform git:(main) ✗ terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
→ terraform git:(main) ✗ terraform validate
Success! The configuration is valid.
```

Figure 5: Initialisation et validation de Terraform

3.4. Création du dépôt Amazon ECR

Le module ECS regroupe les ressources AWS et leurs sorties. Cette séparation évite de mélanger la configuration générale du projet avec les détails du déploiement Fargate.

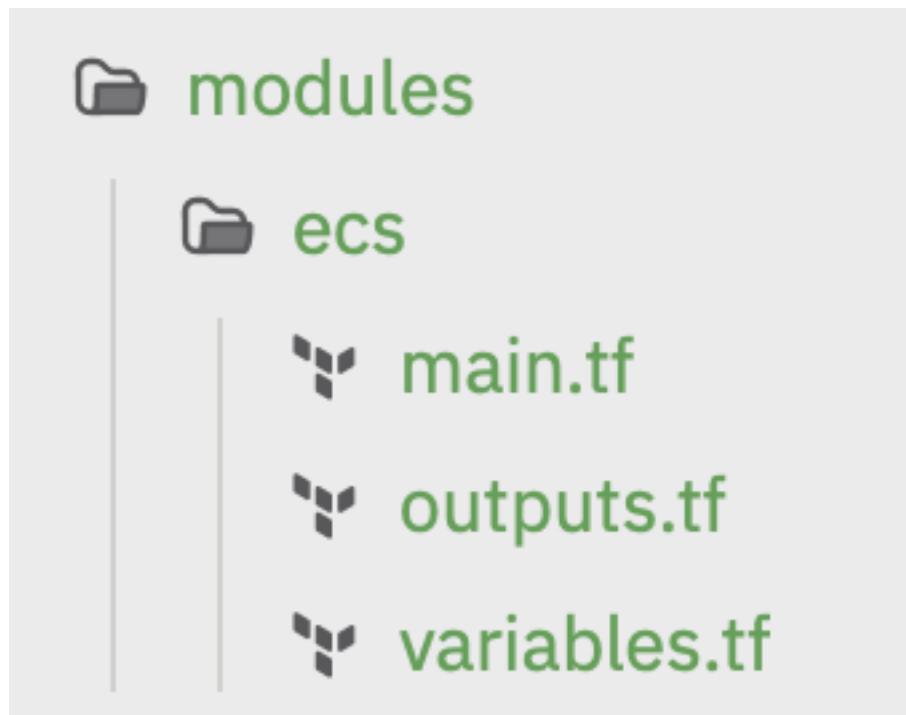


Figure 6: Arborescence du projet et du module Terraform ECS

```
→ PROJET git:(main) x terraform -chdir=terraform init
Initializing modules...
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0
```

Initializing the backend...

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
→ PROJET git:(main) x terraform -chdir=terraform validate
Success! The configuration is valid.
```

```
→ PROJET git:(main) x terraform -chdir=terraform plan -out=tfplan
```

Terraform used the selected providers to generate the following execution plan.

Resource

actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# module.ecs.aws_ecr_lifecycle_policy.app will be created
+ resource "aws_ecr_lifecycle_policy" "app" {
  + id          = (known after apply)
  + policy      = jsonencode(
    {
      + rules = [
        + {
          + action      = {
            + type = "expire"
          }
          + description = "Conserver les dix dernieres images"
          + rulePriority = 1
          + selection    = {
            + countNumber = 10
            + countType   = "imageCountMoreThan"
            + tagStatus   = "any"
          }
        },
      ]
    }
  )
  + registry_id = (known after apply)
  + repository  = "ipssi-orchestration-app"
}
```

```
# module.ecs.aws_ecr_repository.app will be created
+ resource "aws_ecr_repository" "app" {
  + arn                = (known after apply)
  + force_delete       = true
  + id                 = (known after apply)
  + image_tag_mutability = "IMMUTABLE"
  + name               = "ipssi-orchestration-app"
  + registry_id        = (known after apply)
  + repository_url      = (known after apply)
  + tags_all           = {
    + "ManagedBy" = "Terraform"
    + "Project"    = "ipssi-orchestration"
  }

  + encryption_configuration {
    + encryption_type = "AES256"
    + kms_key         = (known after apply)
  }

  + image_scanning_configuration {
    + scan_on_push = true
  }
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ ecr_repository_url = (known after apply)
```

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
 terraform apply "tfplan"


```

→ PROJET git:(main) ✗ terraform -chdir=terraform init
Initializing modules...
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Initializing the backend...

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
→ PROJET git:(main) ✗ terraform -chdir=terraform validate
Success! The configuration is valid.

→ PROJET git:(main) ✗ terraform -chdir=terraform plan -out=tfplan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# module.ecs.aws_ecr_lifecycle_policy.app will be created
+ resource "aws_ecr_lifecycle_policy" "app" {
  + id          = (known after apply)
  + policy      = jsonencode(
    {
      + rules = [
        + {
          + action      = {
            + type = "expire"
          }
          + description = "Conserver les dix dernieres images"
          + rulePriority = 1
          + selection   = {
            + countNumber = 10
            + countType   = "imageCountMoreThan"
            + tagStatus   = "any"
          }
        }
      ],
    }
  )
  + registry_id = (known after apply)
  + repository  = "ipssi-orchestration-app"
}

# module.ecs.aws_ecr_repository.app will be created
+ resource "aws_ecr_repository" "app" {
  + arn              = (known after apply)
  + force_delete     = true
  + id              = (known after apply)
  + image_tag_mutability = "IMMUTABLE"
  + name            = "ipssi-orchestration-app"
  + registry_id     = (known after apply)
  + repository_url   = (known after apply)
  + tags_all        = {
    + "ManagedBy" = "Terraform"
    + "Project"    = "ipssi-orchestration"
  }

  + encryption_configuration {
    + encryption_type = "AES256"
    + kms_key         = (known after apply)
  }

  + image_scanning_configuration {
    + scan_on_push = true
  }
}

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ecr_repository_url = (known after apply)

```

```

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
terraform apply "tfplan"

```

Figure 7: Premier plan Terraform pour le dépôt ECR

Terraform crée le dépôt ECR ipssi-orchestration-app. Ses tags sont immuables. Une nouvelle image ne peut donc pas écraser une version déjà publiée avec le même tag. ECR analyse chaque image lors du push et utilise le chiffrement AES-256 géré par AWS.

La politique de cycle de vie conserve les dix images les plus récentes. Pendant terraform apply, une ressource terraform_data construit l'image locale et l'envoie dans ECR avec le hash du commit. La définition de tâche ECS reprend ensuite cette adresse complète.

3.5. Ajout du cluster ECS et du réseau

Le cluster ipssi-orchestration-cluster regroupe les tâches du projet. Container Insights relève leur consommation. Le pilote awslogs envoie les sorties du conteneur dans CloudWatch, où elles restent disponibles pendant sept jours.

Le module recherche le VPC et les sous-réseaux par défaut du compte AWS Academy. Fargate attribue une adresse IP publique à chaque tâche. Le Security Group ouvre le port 8080 uniquement pour l'adresse IPv4 fournie au pipeline. Il ne contient aucune autre règle entrante.

```
→ PROJET git:(main) ✗ terraform -chdir=terraform show -no-color ecs.tfplan | tail -25
+ owner_id              = (known after apply)
+ revoke_rules_on_delete = false
+ tags_all              = {
  + "ManagedBy" = "Terraform"
  + "Project"    = "ipssi-orchestration"
}
+ vpc_id                = "vpc-0b5c31d7ddaa30059"
}

# module.ecs.terraform_data.app_image will be created
+ resource "terraform_data" "app_image" {
  + id              = (known after apply)
  + triggers_replace = [
    + "v1",
    + "94e4449e9578c1354dc7572effc296d083fe6b6b063cbdd824e196aa29d45573",
  ]
}
```

Plan: 10 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ ecr_image_uri      = (known after apply)
+ ecr_repository_url = (known after apply)
+ ecs_cluster_name   = "ipssi-orchestration-cluster"
+ ecs_service_name   = "ipssi-orchestration-service"
```

Figure 8: Plan Terraform ECS : dix ressources à créer

DescribeClusters				
ContainerInsights	Nom	Services	Statut	Taches
enabled	ipssi-orchestration-cluster	1	ACTIVE	0

Figure 9: Clusters

DescribeLogGroups		
Nom	Retention	
/ecs/ipssi-orchestration	7	

Figure 10: Durée de rétention

DescribeSecurityGroups		
Description	Acces HTTP limite au poste de demonstration	
Nom	ipssi-orchestration-ecs	
Port	8080	
Protocole	tcp	
SourceAutorisee	146.75.166.55/32	

Figure 11: Security Group limité à une adresse IP en /32

3.6. Définition de tâche Fargate

La définition de tâche réserve 256 unités de CPU et 512 Mio de mémoire. Elle précise aussi l'architecture ARM64, car Docker Desktop construit l'image sur un Mac Apple Silicon.

ECS utilise LabRole pour télécharger l'image ECR et écrire dans CloudWatch. Ce choix vient des restrictions AWS Academy, qui empêchent de créer un rôle IAM dédié. Le conteneur expose le port 8080 et ECS exécute son contrôle de santé sur /health.

DescribeTaskDefinition		
CPU	256	
Famille	ipssi-orchestration-task	
Image	935301205663.dkr.ecr.us-east-1.amazonaws.com/ipssi-orchestration-app:6c9f0c85aa65	
Memoire	512	
Port	8080	
Revision	1	
RoleExecution	arn:aws:iam::935301205663:role/LabRole	

Figure 12: Task Definition : image ECR, ressources, port 8080 et rôle LabRole

```
{
  "command": [
    "CMD-SHELL",
    "node -e \"fetch('http://127.0.0.1:8080/health').then(r => process.exit(r.ok ? 0 : 1)).catch(() => process.exit(1))\""
  ],
  "interval": 30,
  "timeout": 5,
  "retries": 3,
  "startPeriod": 10
}
```

Figure 13: Configuration du Health Check

3.7. Service ECS et autoscaling

Le service demande en permanence deux tâches Fargate. Lorsqu'une tâche s'arrête, ECS en lance une autre. Le circuit breaker marque le déploiement en échec si la nouvelle version n'atteint pas un état stable.

La politique d'autoscaling autorise entre deux et quatre tâches. Elle vise une consommation CPU moyenne de 60 %.

DescribeServices	
Actives	2
Desirees	2
EnAttente	0
Nom	ipssi-orchestration-service
Statut	ACTIVE
TaskDefinition	arn:aws:ecs:us-east-1:935301205663:task-definition/ipssi-orchestration-task:2

Figure 14: Service ECS avec deux tâches en fonctionnement

DescribeServices			
Actives	Desirees	EnAttente	Statut
2	2	0	ACTIVE

Figure 15: Service ECS

```
→ PROJÉT git:(main) aws application-autoscaling describe-scalable-targets \
--service-namespace ecs \
--resource-ids service/ipssi-orchestration-cluster/ipssi-orchestration-service \
--query 'ScalableTargets[0].{Ressource:ResourceId,Minimum:MinCapacity,Maximum:MaxCapacity}' \
--output table \
--no-cli-pager
```

DescribeScalableTargets			
Maximum	Minimum	Ressource	
4	2	service/ipssi-orchestration-cluster/ipssi-orchestration-service	

```
→ PROJÉT git:(main) aws application-autoscaling describe-scaling-policies \
--service-namespace ecs \
--resource-id service/ipssi-orchestration-cluster/ipssi-orchestration-service \
--query 'ScalingPolicies[0].{Nom:PolicyName,CibleCPU:TargetTrackingScalingPolicyConfiguration.TargetValue,ScaleOut:TargetTrackingScalingPolicyConfiguration.ScaleOutCooldown,ScaleIn:TargetTrackingScalingPolicyConfiguration.ScaleInCooldown}' \
--output table \
--no-cli-pager
```

DescribeScalingPolicies			
CibleCPU	Nom	ScaleIn	ScaleOut
60.0	ipssi-orchestration-cpu-target	120	60

Figure 16: Configuration de l'autoscaling ECS entre deux et quatre tâches

3.8. Vérification du déploiement ECS

Après le déploiement, nous avons retrouvé l'adresse IP publique à partir de l'interface réseau d'une tâche. La réponse HTTP indiquait Amazon ECS Fargate, ce qui confirme que la requête atteignait le conteneur déployé sur ECS.

```

→ PROJET git:(main) curl --connect-timeout 5 "http://${TP_ECS_IP}:8080/"
{"application":"orchestration-demo","version":"1b0dc79e1bdd","platform":"Amazon ECS Fargate","instance":"ip-172-31-5-98.ec2.internal"}
→ PROJET git:(main) curl --connect-timeout 5 "http://${TP_ECS_IP}:8080/health"
{"status":"ok"}

```

Figure 17: Réponse HTTP obtenue depuis une tâche ECS

Nous avons ensuite arrêté une tâche à la main. ECS l'a remplacée par une tâche avec un nouvel ARN et le service est revenu à deux tâches actives.

```

→ PROJET git:(main) TP_OLD_TASK=$(aws ecs list-tasks \
--cluster ipssi-orchestration-cluster \
--service-name ipssi-orchestration-service \
--query 'taskArns[0]' \
--output text)
→ PROJET git:(main) echo "Tache arretee volontairement : $TP_OLD_TASK"
Tache arretee volontairement : arn:aws:ecs:us-east-1:935301205663:task/ipssi-orchestration-cluster/5202f160073b4a62b13d202a4c7dedcd
→ PROJET git:(main) aws ecs stop-task \
--cluster ipssi-orchestration-cluster \
--task "$TP_OLD_TASK" \
--reason "Test auto-reparation" \
--query 'task.{Tache:taskArn,Statut:lastStatus}' \
--output table \
--no-cli-pager
+-----+-----+
|                                         StopTask                                         |
+-----+-----+
| Statut | Tache |
+-----+-----+
| RUNNING | arn:aws:ecs:us-east-1:935301205663:task/ipssi-orchestration-cluster/5202f160073b4a62b13d202a4c7dedcd |
+-----+-----+
→ PROJET git:(main) aws ecs wait services-stable \
--cluster ipssi-orchestration-cluster \
--services ipssi-orchestration-service
→ PROJET git:(main) echo "Ancienne tache : $TP_OLD_TASK"
echo "Taches actuellement actives :"
Ancienne tache : arn:aws:ecs:us-east-1:935301205663:task/ipssi-orchestration-cluster/5202f160073b4a62b13d202a4c7dedcd
Taches actuellement actives :
→ PROJET git:(main) aws ecs list-tasks \
--cluster ipssi-orchestration-cluster \
--service-name ipssi-orchestration-service \
--query 'taskArns' \
--output table \
--no-cli-pager
+-----+-----+
|                                         ListTasks                                         |
+-----+-----+
| arn:aws:ecs:us-east-1:935301205663:task/ipssi-orchestration-cluster/a7164943ad4f44768ab0f1f6f89db612 |
| arn:aws:ecs:us-east-1:935301205663:task/ipssi-orchestration-cluster/d4e6c4fedbfd49c28c2b51a247c52290 |
+-----+-----+
→ PROJET git:(main) aws ecs describe-services \
--cluster ipssi-orchestration-cluster \
--services ipssi-orchestration-service \
--query 'services[0].{Desirees:desiredCount,Actives:runningCount,EnAttente:pendingCount}' \
--output table \
--no-cli-pager
+-----+-----+
| DescribeServices |
+-----+-----+
| Actives | Desirees | EnAttente |
+-----+-----+
| 2 | 2 | 0 |
+-----+-----+

```

Figure 18: Remplacement automatique d'une tâche ECS arrêtée

4. Déploiement sur Kubernetes avec Terraform

4.1. Préparation de Minikube

Le provider Terraform kubernetes se connecte au contexte minikube. L'addon ingress fournit le contrôleur NGINX. metrics-server mesure l'utilisation du processeur dont le HPA a besoin.

Minikube ne peut pas tirer directement l'image privée sans configuration ECR. Terraform charge donc dans Minikube l'image construite localement. ECS et Kubernetes exécutent le même code avec le même tag de commit.

```

→ PROJÉT git:(main) minikube -p minikube status
minikube
type: Control Plane
host: Running
kubelét: Running
apiserver: Running
kubeconfig: Configured

→ PROJÉT git:(main) kubectl --context minikube get nodes -o wide
NAME      STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE           KERNEL-VERSION   CONTAINER-RUNTIME
minikube  Ready     control-plane  47h   v1.35.1   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm) 6.12.76-linuxkit docker://29.2.1

```

Figure 19: Cluster Minikube démarré et nœud dans l'état Ready

4.2. Deployment, ConfigMap et service

Le namespace orchestration-demo contient toutes les ressources de l'application. La ConfigMap transmet APP_VERSION et PLATFORM aux conteneurs.

Le Deployment démarre deux pods et fixe leurs ressources CPU et mémoire. Les sondes de disponibilité et de vivacité interrogent /health. Le Service ClusterIP répartit les requêtes entre les pods, puis l'Ingress NGINX associe le nom demo.local à ce service.

```

→ PROJÉT git:(main) kubectl --context minikube \
  -n orchestration-demo \
  get configmap,deployment,pods,service,ingress,hpa
NAME      DATA  AGE
configmap/kube-root-ca.crt  1      46m
configmap/orchestration-demo-config  2      46m

NAME      READY  UP-TO-DATE  AVAILABLE  AGE
deployment.apps/orchestration-demo  2/2    2           2          46m

NAME      READY  STATUS    RESTARTS  AGE
pod/orchestration-demo-6887568475-cmmw6  1/1    Running    0         28m
pod/orchestration-demo-6887568475-dpwqc  1/1    Running    0         27m

NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/orchestration-demo  ClusterIP    10.106.136.61  <none>        80/TCP     46m

NAME      CLASS  HOSTS      ADDRESS      PORTS    AGE
ingress.networking.k8s.io/orchestration-demo  nginx    demo.local   192.168.49.2  80      46m

NAME      REFERENCE      TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
horizontalpodautoscaler.autoscaling/orchestration-demo  Deployment/orchestration-demo  cpu: 1%/60%  2        6        2         46m
→ PROJÉT git:(main) kubectl --context minikube \
  -n orchestration-demo \
  rollout status deployment/orchestration-demo
deployment "orchestration-demo" successfully rolled out

```

Figure 20: Deployment, pods, ConfigMap, Service et Ingress

```

→ PROJÉT git:(main) curl http://127.0.0.1:8082/
echo
curl http://127.0.0.1:8082/health
echo
{"application": "orchestration-demo", "version": "1b0dc79e1bdd", "platform": "Kubernetes Minikube", "instance": "orchestration-demo-6887568475-cmmw6"}
{"status": "ok"}

```

Figure 21: Réponse HTTP Kubernetes sur le port-forward local 8082

4.3. Autoscaling Kubernetes

Le Horizontal Pod Autoscaler autorise entre deux et six pods et vise 60 % d'utilisation CPU. Pour le tester, dix pods temporaires ont appelé la route /cpu en boucle. La mesure a atteint 442 % et le HPA a porté le Deployment de deux à quatre pods.

```

→ PROJÉT git:(main)
  kubectl --context minikube \
    -n orchestration-demo \
    get hpa
NAME      REFERENCE      TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
orchestration-demo  Deployment/orchestration-demo  cpu: 1%/60%  2        6        2         66m
→ PROJÉT git:(main) kubectl --context minikube \
  -n orchestration-demo \
  top pods
NAME      CPU(cores)  MEMORY(bytes)
orchestration-demo-6887568475-cmmw6  1m          9Mi
orchestration-demo-6887568475-dpwqc  1m          9Mi

```

Figure 22: HPA avant la génération de charge

Every 5.0s: kubectl --context minikube -n orchestration-demo get hpa,pods

heavensandearth.local: Fri Aug 28 16:47:14 2026

in 0.069s (0)

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICA
S AGE					
horizontalpodautoscaler.autoscaling/orchestration-demo	Deployment/orchestration-demo	cpu: 442%/60%	2	6	4
68m					

NAME	READY	STATUS	RESTARTS	AGE
pod/load-generator-649c4d6c79-4pdh4	1/1	Running	0	90s
pod/load-generator-649c4d6c79-92kb4	1/1	Running	0	90s
pod/load-generator-649c4d6c79-bnf96	1/1	Running	0	90s
pod/load-generator-649c4d6c79-cwklk	1/1	Running	0	90s
pod/load-generator-649c4d6c79-dn4rl	1/1	Running	0	90s
pod/load-generator-649c4d6c79-hzszc	1/1	Running	0	90s
pod/load-generator-649c4d6c79-jhlvd	1/1	Running	0	90s
pod/load-generator-649c4d6c79-mtd6f	1/1	Running	0	90s
pod/load-generator-649c4d6c79-s5pjr	1/1	Running	0	90s
pod/load-generator-649c4d6c79-tkdrn	1/1	Running	0	94s
pod/orchestration-demo-6887568475-8hk72	1/1	Running	0	27s
pod/orchestration-demo-6887568475-cmmw6	1/1	Running	0	50m
pod/orchestration-demo-6887568475-dpwqc	1/1	Running	1 (37s ago)	50m
pod/orchestration-demo-6887568475-kx9b5	1/1	Running	0	27s

Figure 23: Augmentation du nombre de pods pendant la charge

4.4. Sécurité Kubernetes

Le conteneur utilise un compte non privilégié. Sa configuration interdit l'élévation de droits, supprime les capacités Linux et monte le système de fichiers racine en lecture seule. Kubernetes ne lui fournit pas de jeton de ServiceAccount.

Une NetworkPolicy filtre le trafic entrant. La ValidatingAdmissionPolicy exige un tag explicite et refuse latest. Le test avec nginx:latest a bien été bloqué avant la création du pod.

```
→ PROJET git:(main) kubectl --context minikube \
  -n orchestration-demo \
  delete deployment load-generator
deployment.apps "load-generator" deleted from orchestration-demo namespace
→ PROJET git:(main) kubectl --context minikube \
  -n orchestration-demo \
  run forbidden-latest \
  --image=nginx:latest
The pods "forbidden-latest" is invalid: : ValidatingAdmissionPolicy 'ipssi-orchestration-immutable-tags' with binding 'ipssi-orchestration-immutable-tags' denied request: Chaque conteneur doit utiliser un tag différent de latest.
```

Figure 24: Refus de la création d'un pod utilisant nginx:latest

4.5. Auto-réparation Kubernetes

Nous avons supprimé un des deux pods. Le ReplicaSet en a créé un autre, avec un nom différent, puis le Deployment est revenu à deux pods Running.

```
→ PROJET git:(main) echo "Pod qui sera supprimer : $TP_OLD_POD"
Pod qui sera supprimer : orchestration-demo-6887568475-84nx1
→ PROJET git:(main) kubectl --context minikube \
  -n orchestration-demo \
  delete pod "$TP_OLD_POD"
pod "orchestration-demo-6887568475-84nx1" deleted from orchestration-demo namespace
→ PROJET git:(main) kubectl --context minikube \
  -n orchestration-demo \
  wait --for=condition=available \
  deployment/orchestration-demo \
  --timeout=120s
deployment.apps/orchestration-demo condition met
→ PROJET git:(main) echo "Pod supprimer : $TP_OLD_POD"

kubectl --context minikube \
  -n orchestration-demo \
  get pods \
  -l app=orchestration-demo \
  -o wide
Pod supprimer : orchestration-demo-6887568475-84nx1
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS
GATES
orchestration-demo-6887568475-dpwqc 1/1 Running 1 (10m ago) 60m 10.244.120.117 minikube <none> <none>
orchestration-demo-6887568475-wj55p 1/1 Running 0 2m3s 10.244.120.72 minikube <none> <none>
```

Figure 25: Suppression d'un pod et création automatique de son remplaçant

5. Automatisation avec Jenkins

5.1. Configuration du pipeline

Le fichier `Jenkinsfile` décrit le pipeline. Jenkins injecte les trois valeurs temporaires AWS depuis son gestionnaire de credentials. Le dépôt Git ne contient aucune de ces valeurs.

Au début du build, Jenkins lit les douze premiers caractères du hash Git. Ce hash devient le tag de l'image sur ECR et dans Minikube. Une capture permet donc d'associer une image au commit qui l'a produite.

Le pipeline exécute ces étapes :

- Checkout récupère le dépôt Git ;
- Preflight vérifie AWS, Docker et Minikube ;
- Validate initialise et valide Terraform ;
- Plan génère et archive le plan ;
- Approve attend une approbation humaine ;
- Apply applique le fichier de plan approuvé ;
- Verify vérifie la disponibilité des deux plateformes.

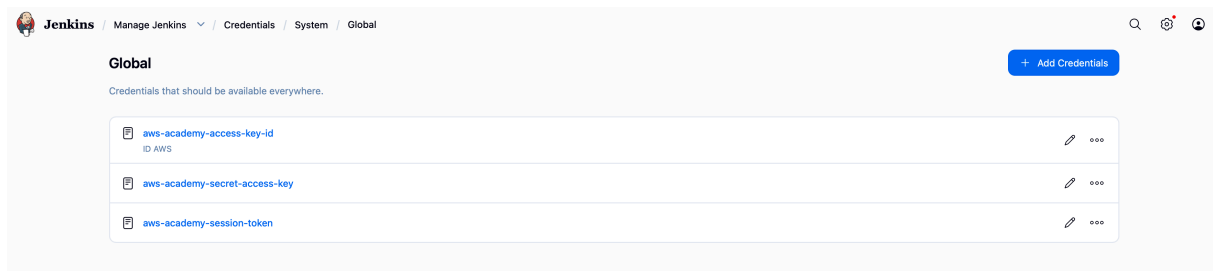


Figure 26: Création des credentials dans Jenkins

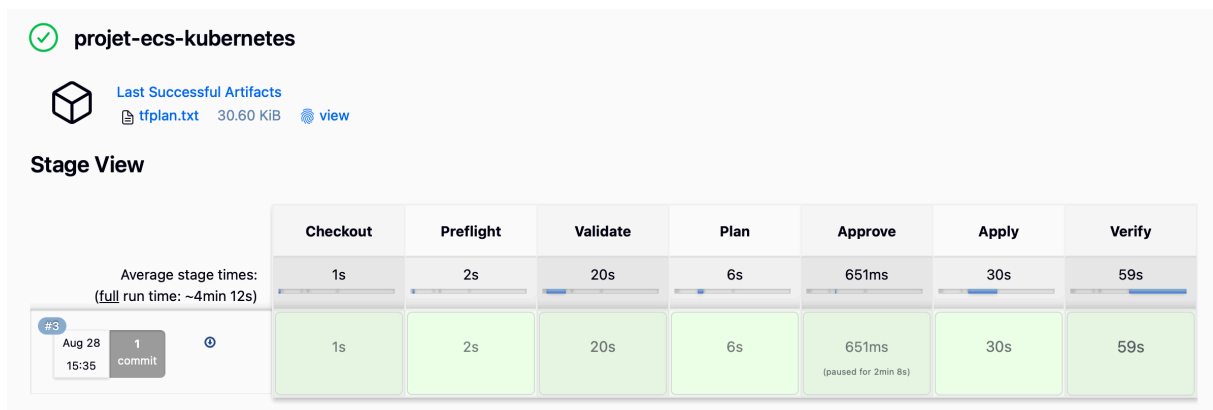


Figure 27: Déploiement via Jenkins

5.2. Idempotence

Nous avons relancé le même commit avec les mêmes paramètres. Terraform a affiché `No changes`, car l'état réel correspondait déjà aux fichiers du dépôt. Le second passage n'a donc recréé aucune ressource.


```

[0m[1m[32mNo changes.[0m[1m Your infrastructure matches the configuration.[0m

[0mTerraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.
+ terraform show -no-color tfplan
[Pipeline] }
[Pipeline] // dir
[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Approve)
[Pipeline] timeout
Timeout set to expire in 20 min
[Pipeline] {
[Pipeline] input
Appliquer le plan sur ECS et Kubernetes ?
Appliquer or Abort

```

Figure 28: Second plan Jenkins indiquant No changes

6. Difficultés rencontrées

6.1. Conflit de ports

Jenkins occupait déjà le port local 8080. Notre premier curl a donc ouvert la page de connexion Jenkins au lieu de joindre l'application. Nous avons déplacé le test Docker sur 8081 et le port-forward Kubernetes sur 8082. Le port du conteneur reste 8080 sur les deux plateformes.

6.2. Identifiants AWS temporaires

AWS Academy délivre des identifiants temporaires. Après leur expiration, AWS a renvoyé `InvalidClientTokenId`, puis une politique `voc-cancel-cred` a refusé les appels du pipeline. Nous avons relancé le lab et remplacé les trois valeurs enregistrées dans Jenkins.

6.3. Contraintes AWS Academy

Le lab bloque la création de rôles IAM et d'un cluster EKS. Nous avons donc réutilisé `LabRole` pour ECS et installé Kubernetes localement avec Minikube.

7. Bilan des tests

Test réalisé	Résultat attendu	Résultat obtenu	Statut
Validation Terraform	La configuration est valide	terraform validate termine avec succès	Réussi
Déploiement ECS	Deux tâches Fargate actives	Deux tâches désirées et deux tâches actives	Réussi
Accès HTTP ECS	L'application répond sur le port 8080	Le serveur a répondu depuis l'adresse publique de la tâche	Réussi

Test réalisé	Résultat attendu	Résultat obtenu	Statut
Autoréparation ECS	Une tâche arrêtée est remplacée	ECS a créé une nouvelle tâche et retrouvé deux tâches actives	Réussi
Déploiement Kubernetes	Deux pods sont disponibles	Les deux pods sont dans l'état Running	Réussi
Accès HTTP Kubernetes	L'application répond sur le port local 8082	Le serveur a répondu à travers le port-forward	Réussi
Autoscaling Kubernetes	Le nombre de pods augmente pendant la charge	Avec une CPU mesurée à 442 %, le HPA est passé de deux à quatre pods	Réussi
Politique de sécurité	Les images utilisant le tag latest sont refusées	La politique d'admission a bloqué l'image nginx:latest	Réussi
Autoréparation Kubernetes	Un pod supprimé est automatiquement remplacé	Un nouveau pod a remplacé celui supprimé	Réussi
Pipeline Jenkins	Les étapes du pipeline sont exécutées	Checkout, Validate, Plan, Approve, Apply et Verify ont réussi	Réussi
Idempotence Terraform	Un second plan ne propose aucune modification	Le second plan Jenkins a affiché No changes	Réussi

Tous les tests prévus ont réussi. ECS et Kubernetes ont conservé le nombre d'instances demandé après un arrêt volontaire. Le test de charge a aussi déclenché le HPA Kubernetes. Enfin, le second plan Terraform n'a proposé aucune modification.

Jenkins enchaîne la validation, le plan et le déploiement. L'étape Approve laisse toutefois la main à l'utilisateur avant toute modification de l'infrastructure.

8. Conclusion

Nous avons déployé la même application Node.js sur ECS Fargate et sur Kubernetes à partir d'un seul dépôt. Terraform décrit les deux environnements. Jenkins exécute le même enchaînement à chaque commit et attend notre accord avant `terraform apply`.

Les tests ont surtout permis de vérifier le comportement des orchestrateurs, pas seulement la création des ressources. ECS a remplacé une tâche arrêtée. Kubernetes a remplacé un pod supprimé et a augmenté le nombre de réplicas pendant la charge. La politique d'admission a aussi refusé `nginx:latest` comme prévu.

AWS Academy a imposé deux choix visibles dans l'architecture. ECS utilise LabRole et Kubernetes tourne sur Minikube au lieu d'EKS. Pour une mise en production, il faudrait au minimum ajouter un Load Balancer HTTPS, stocker l'état Terraform dans un backend distant et remplacer les credentials temporaires du lab par une authentification adaptée à Jenkins.